Module 06 Lab - Regression and Classification Models**Objective:** To understand the difference between regression and classification and to build your first linear models for both tasks.**In this lab, you will write the code to train and evaluate the models.**

Part 1: Regression vs. ClassificationThis is the most fundamental distinction between types of supervised learning problems.* **Regression:** The goal is to predict a **continuous numerical value**. * *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.* **Classification:** The goal is to predict a **discrete category or class label**. * *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.**In this lab, we will tackle one of each.**

Part 2: Linear Regression**Concept:** Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".* **Problem:** We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
  # Load and prepare the data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
  # For simplicity, we'll drop rows with missing age
df.dropna(subset=['Age'], inplace=True)
  # Define features and target
features = ['Age', 'Pclass']
target_reg = 'Fare'
X_reg = df[features]
y_reg = df[target_reg]
  # Split the data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)
```

Task 1: Train and Evaluate a Linear Regression Model**Your Task:**1. Create an instance of the `LinearRegression` model.2. Train the model using the training data (`X_train_reg`, `y_train_reg`).3. Make predictions on the test data.4. Evaluate the model using `mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

```
# --- ENTER YOUR CODE HERE ---
import numpy as np
# 1. Create the model instance# lr_model = ...
lr_model = LinearRegression()
# 2. Train the model# lr_model.fit(...)
lr_model.fit(X_train_reg, y_train_reg)
# 3. Make predictions# y_pred_reg = lr_model.predict(...)
y_pred_reg = lr_model.predict(X_test_reg)
# 4. Evaluate the model# mse = mean_squared_error(y_test_reg, y_pred_reg)
mse = mean_squared_error(y_test_reg, y_pred_reg)
print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
print(f"The square root of this is {np.sqrt(mse):.2f}")
```

```
Mean Squared Error for Fare Prediction: 3364.92
The square root of this is 58.01
```

Part 3: Logistic Regression**Concept:** Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.* **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
# Prepare data for classification
# We need to encode 'Sex' column
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
# Define features and target
features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'
X_cls = df[features_cls]
y_cls = df[target_cls]
 # Split the data
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(X_cls, y_cls, test_size=0.2, random_state=42)
```

Task 2: Train and Evaluate a Logistic Regression Model**Your Task:**1. Create an instance of the `LogisticRegression` model.2. Train the model using the classification training data.3. Make predictions on the test data.4. Evaluate the model using `accuracy_score`.

```
# --- ENTER YOUR CODE HERE ---
# 1. Create the model instance
log_model = LogisticRegression(max_iter=1000)
# 2. Train the model
log_model.fit(X_train_cls, y_train_cls)
# 3. Make predictions
y_pred_cls = log_model.predict(X_test_cls)
# 4. Evaluate the model
accuracy = accuracy_score(y_test_cls, y_pred_cls)
print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

```
Accuracy for Survival Prediction: 74.83%
```

📝 Knowledge Check**Instructions:** Answer the following questions in this markdown cell.1. **In your own words, what is the key difference between a regression problem and a classification problem?**2. **The `LinearRegression` model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent?**3. **Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model?** Why wouldn't accuracy be a good metric for the fare prediction task?

1. Regression predicts numerical values, such as the price of a Titanic passenger's ticket. Classification is used to predict categories, such as whether a passenger survived or did not survive.
2. The coefficients show how Age and Pclass affect the predicted ticket fare. The Age coefficient is approximately negative 0.44, meaning the predicted fare decreases by about 44 cents for each additional year of age. The Pclass coefficient is approximately negative 38.30, meaning the predicted fare decreases by about 38 dollars and 30 cents when the passenger class number increases by one. Both coefficients are negative, showing a decrease in the predicted fare when either value increases, while the other stays the same.
3. We used mean squared error for regression because we were predicting numerical values, such as ticket fares and needed to measure the differences between the predicted and actual prices. We used accuracy for classification because we were predicting whether passengers survived or not and accuracy tells us how many predictions were correct. Accuracy would not be a good measurement for ticket fares because the prices are continous values and it would not show how close or far off our predictions were.

```
print(lr_model.coef_)
```

```
[ -0.43695889 -38.29728935]
```